

Lecture – 8

Advanced Sorting Methods –Part 1

M.G.Noel A.S Fernando (PhD)

Professor

NSBM/Dept. of Information Systems Engineering, UCSC



Sorting Algorithms

- We learned basic algorithms in Year 1, such as sorting by exchange (e.g., Bubble Sort), sorting by insertion (e.g., Insertion Sort), and sorting by selection (e.g., Selection Sort).
- In advanced sorting methods, we study two categories:
- **Comparison-Based Sorting Algorithms** (e.g., Merge Sort, Shell Sort, Quick Sort, Heap Sort).
- **Non-Comparison-Based Sorting Algorithms** (e.g., Counting Sort, Radix Sort, Bucket Sort).

Merge Sort

- **Type:** Comparison-based, Divide-and-Conquer algorithm
- Procedure :Merge Sort is a **divide-and-conquer sorting algorithm**. It works by repeatedly splitting the list into smaller parts, sorting those parts, and then merging them back together in sorted order.

Method 1- Steps of Merge Sort:

Step 1: Divide

- Split the unsorted list into two halves (left and right).
- Keep dividing until each sublist has only one element (a single element is already sorted).

Step 2: Conquer (Sort recursively):

- Recursively apply merge sort to each half.

Step 3: Merge:

- Combine (merge) the two sorted halves into a single sorted list.
- This is done by repeatedly comparing the smallest elements from each half and appending the smaller one.

Algorithm:

mergeSort(arr[], l, r)

If $l < r$

1. Find the middle point to divide the array into two halves: middle $m = (l+r)/2$
2. Call mergeSort for first half: Call mergeSort(arr, l, m)
3. Call mergeSort for second half: Call mergeSort(arr, m+1, r)
4. Merge the two halves sorted in step 2 and 3: Call merge(arr, l, m, r)

Question 1

- Consider the following data set:
- 38,27,43,3,9,82,10
- Using a suitable diagram, explain how to sort the above dataset using the **Merge Sort algorithm**.
- Write C code to implement the Merge Sort algorithm

Method 2-Merge Sort

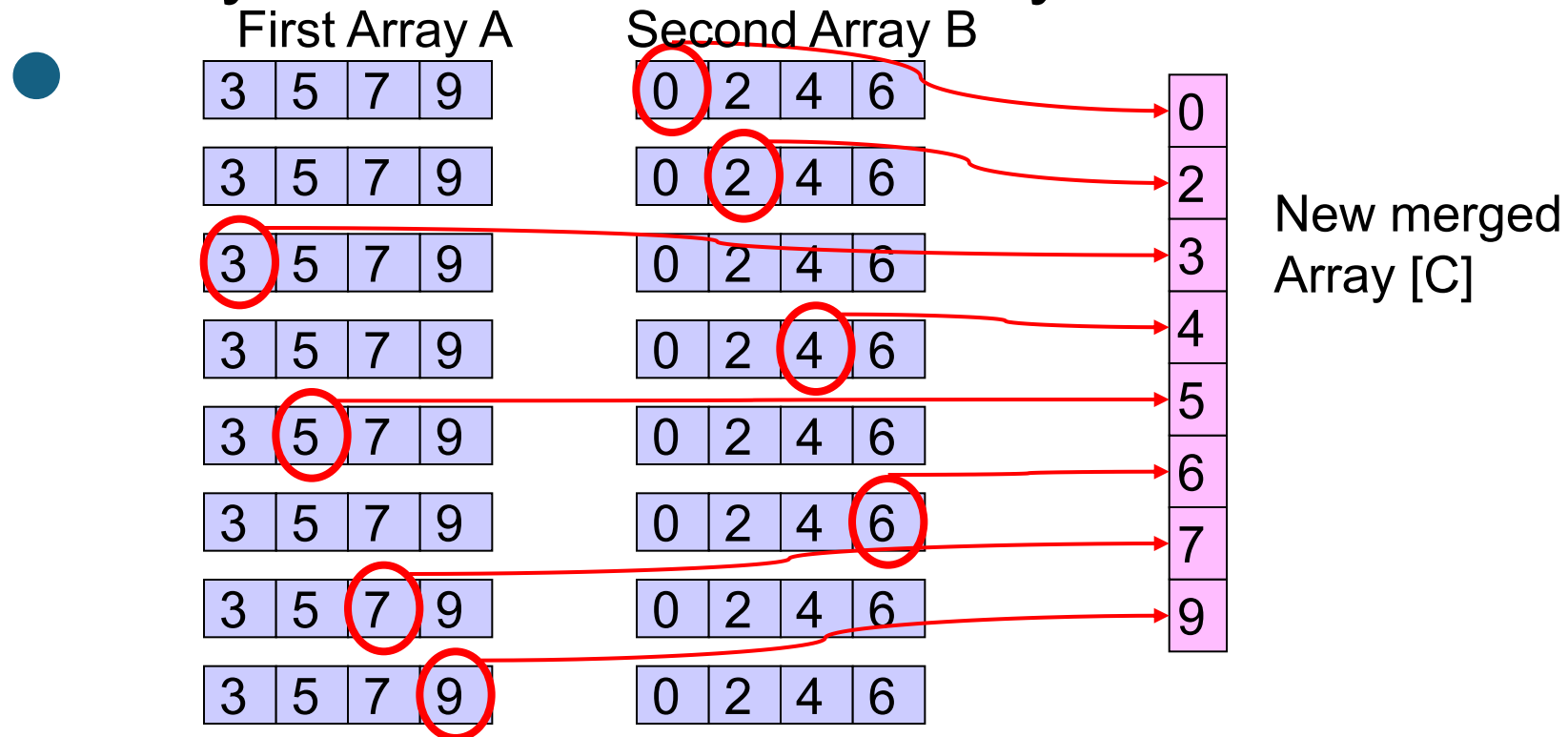
Merge sort Algorithm

- **Merging**

- Merging is the process of combining two or more sorted file into a third sorted file.
- The basic merge algorithm takes two input arrays A and B, output array C, and three counters Actr, Bctr, Cctr which is initially set the beginning of their respective arrays.
- The smaller of $A[Actr]$, $B[Bctr]$ is copy to the next entry in C, and the appropriated counters are advanced. When either input array is exhausted, the rest of the other array is copy to C.

Merge sort

- Merge function — Merging two sorted arrays into one sorted array.



Merge Sort (Cont..)

- If the array A contains 3,5,7,9 and Array B contain 0,2,4,6, then the algorithm proceed as follows :
- First comparison is done between 0 and 3, then 0 is added to C ,then 2 and 3 , then 2 is added etc.

Merge sort - pseudo code Algorithm

/* A_n and B_n are the maximum number of elements of array A and B.*/

Actr:=0;

Bctr:=0;

Cctr:=0;

While ($Actr \leq A_n - 1$) and ($Bctr \leq B_n - 1$)

{

If $A[Actr] < B[Bctr]$ {

$C[Cctr] = A[Actr]$

Actr++;

}

else

Merge sort - pseudo code Algorithm(cont..)

```
C[Cctr]=B[Bctr]
Bctr++;
}
Cctr++;
}
/* copy the remaining elements
While Actr < An-1
/* Copy remaining elements from B */
{
C[Cctr]=A[Actr];
Actr++;
Cctr++;
}
```

Merge sort - pseudo code Algorithm(cont..)

```
While Bctr ≤ Bn-1
/* copy remaining elements from B */
{
C[Cctr]= B[Bctr];
Bctr++;
Cctr++;
}
}
```

Straight Merge Sort

- Divide the file into n sub files of size 1 and merge adjacent pairs of files.
- Then we have approximately $n/2$ files of size 2.
- Repeat this process until there is only one file remaining of size n .

Example

25,57,48,37,12,92,86,33

Original file is divided into n sub-files.

[25],[57],[48],[37],[12],[92],[86],[33]

Pass 1

[25,57],[37,48],[12,92],[33,86]

Pass 2

[25,37,48,57],[12, 33, 86 92]

Pass 3

[12,25,33,37,48,57,86,92]

Entire file is sorted.

Shell Sort

- Shell Sort is an **in-place comparison-based sorting algorithm**.
- It is basically an improvement of **Insertion Sort** that allows the exchange of far-apart elements to reduce the total number of shifts.
- In Insertion Sort, elements are moved only one position at a time → slow for large, unsorted arrays.
- Shell Sort improves this by comparing and swapping elements that are far apart.
- This method sorts separate sub-files of the original file. These sub-files contained K element of the original file.
- The value of k is called increment or gap .

Shell Sort (Cont..)

- For example,
- Original file : $x[1], x[2], x[3], x[4], \dots, x[15]$
- If $k=5$ then
- Sub file 1 : $x[1], x[6], x[11], \dots$
- Sub file 2 : $x[2], x[7], x[12], \dots$
- Sub file 3 : $x[3], x[8], x[13], \dots$
- Sub file 4 : $x[4], x[9], x[14], \dots$
- Sub file 5 : $x[5], x[10], x[15], \dots$
- There are five sub-files, each containing one fifth elements of the original file.

Shell Sort (Cont..)

- In general I^{th} element of the J^{th} sub file is $x[(i-1)*k+j]$
- For example, if the original file is 25,57,48,37,12,92,86,33 and the sequence of increments (4,2,1) are chosen.
- For the first iteration (Increment = 4)
- We can divide the original file into four sub files.

Shell Sort (Cont..)

- 25, 57, 48, 37, 12, 92, 86, 33
- 1st sub file (x[1],x[5]) [25,12]
- 2nd sub file (x[2],x[6]) [57,92]
- 3rd sub file (x[3],x[7]) [48,86]
- 4th sub file (x[4],x[8]) [37,33]
- 12, 57, 48, 33, 25, 92, 86, 37

Second iteration (increment=2)

- 12,57,48,33,25,92,86,37
- First sub file :($x[1],x[3],x[5],x[7]$)
- (12, 48,25,86)
- Second sub file ::($x[2],x[4],x[6],x[8]$)
- (57, 33,92,37)
- [12,33,25,37,48,57,86,92)

Second iteration (increment=1)

- [12,33,25,37,48,57,86,92)
- Increment =1
- [12,25,33,37,48,57,86,92]
- Sorted file.

Pseudocode Algorithm (Shell Sort with increments/ Gaps 4,2,1)

Algorithm ShellSort(A, n)

Input: Array A of length n

Output: Sorted array A

gap_sequence = [4, 2, 1]

for each gap in gap_sequence do

 for i = gap to n-1 do

 temp = A[i]

 j = i

 while j >= gap and A[j-gap] > temp
 do

 A[j] = A[j-gap]

 j = j - gap

 end while

 A[j] = temp

 end for

end for